

PHASE 3 — DOCKER & CONTAINERIZATION

DAY 14: DOCKER WEEK 3 PROJECT

Topic

Containerized Application Delivery

Objectives

The objective of Day 14 is to demonstrate a complete containerized application delivery setup by combining the Docker concepts and practices learned during the previous days.

The mini-project focuses on:

- Containerized application delivery
- Nginx reverse proxy
- Healthchecks
- Persistent data
- Non-root execution
- Image scanning
- Project documentation using a README

1. Containerized Application Delivery

Containerized application delivery means packaging an application and its required environment into containers so that the application can run consistently.

Docker containers provide isolated environments for application components. In this project, the application, database, and Nginx reverse proxy are managed using Docker Compose.

The application communicates with the PostgreSQL database, while Nginx acts as the reverse proxy in front of the application.

2. Project Architecture

The project contains three main services:

Application

The application runs inside a Python container and listens on port 8000.

Database

PostgreSQL is used as the application's database.

Nginx

Nginx works as a reverse proxy. Requests received on port 8080 are forwarded to the application container on port 8000.

The basic request flow is:

Client → Nginx → Application → PostgreSQL Database

3. Nginx Reverse Proxy

A reverse proxy receives requests from clients and forwards them to backend services.

In this project, Nginx listens on port 80 inside its container and is exposed on port 8080 on the host.

The Nginx configuration forwards requests to:

<http://app:8000>

This allows the user to access the application through:

<http://localhost:8080>

The application and Nginx communicate through the Docker Compose network.

4. Healthchecks

Healthchecks are used to determine whether a containerized service is functioning correctly.

The project contains healthchecks for:

- Application
- PostgreSQL database
- Nginx

The application healthcheck tests whether the application responds on port 8000.

The PostgreSQL healthcheck uses `pg_isready` to verify database availability.

The Nginx healthcheck checks whether Nginx can successfully respond to a local HTTP request.

Healthchecks provide a simple way to monitor the operational state of containers.

5. Persistent Data

Containers are designed to be replaceable, so important application data should not depend only on the container's writable filesystem.

Docker volumes provide persistent storage outside the lifecycle of an individual container.

In this project, the PostgreSQL database uses the named volume:

db-data

The volume is mounted at:

/var/lib/postgresql/data

This allows PostgreSQL data to remain available even if the database container is recreated.

6. Non-Root Execution

Running applications as a non-root user is an important container security practice.

The application container is configured to use:

user: "1000:1000"

This prevents the application process from running as the root user.

Running containers with reduced privileges helps limit the potential impact of a security issue inside the application.

7. Docker Compose Configuration

Docker Compose is used to define and manage the application, database, and Nginx services together.

The Compose configuration defines:

- Application service
- PostgreSQL database service
- Nginx service
- Environment variables
- Networks
- Persistent volume
- Healthchecks

- Port mapping
- Service dependencies

The project uses two networks:

- frontend
- backend

The Nginx service connects to the frontend network.

The database connects to the backend network.

The application connects to both networks so that it can communicate with Nginx and PostgreSQL.

8. Hands-On Project Implementation

Step 1 — Start the Docker Compose Application

The project was started using Docker Compose with:

```
sudo docker compose up -d --force-recreate
```

The command recreated and started the application, database, and Nginx containers.

```
shayan@shayan-VirtualBox:~/day12-compose$ sudo docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE  CR
EATED                                STATUS                                PORTS                                SERVICE  CR
day12-compose-app-1                python:3.12-slim                    "sh -c 'pip install ...'"           app      47
seconds ago Up 44 seconds (healthy)
day12-compose-db-1                 postgres:16                          "docker-entrypoint.s..."           db        48
seconds ago Up 45 seconds (healthy)  5432/tcp
day12-compose-nginx-1              nginx:alpine                         "/docker-entrypoint..."           nginx     47
seconds ago Up 43 seconds (health: starting) 0.0.0.0:8080->80/tcp, [::]:808
0->80/tcp
shayan@shayan-VirtualBox:~/day12-compose$
```

The screenshot shows the three project containers being started successfully.

9. Verify Container Status

The running services were checked using:

```
sudo docker compose ps
```

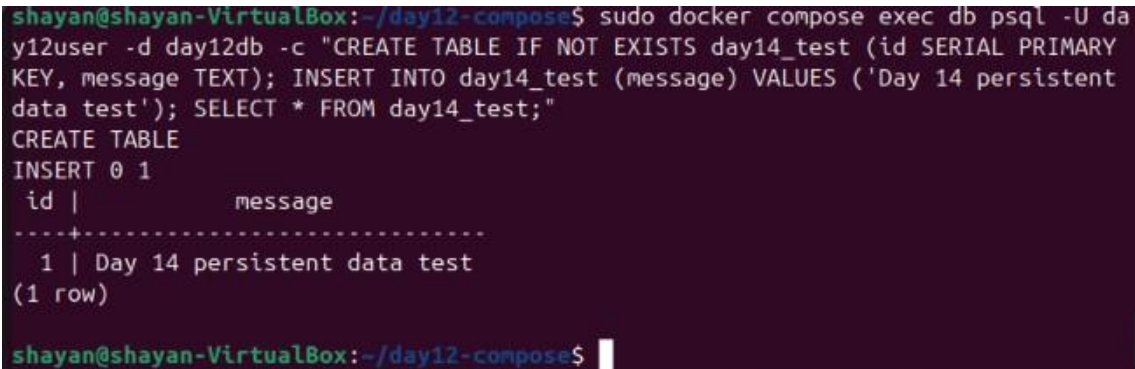
The application and database containers were shown as healthy, and Nginx was monitored until its healthcheck completed successfully.

10. Verify Persistent Database Data

A test record was inserted into the PostgreSQL database using:

```
sudo docker compose exec db psql -U day12user -d day12db -c "CREATE TABLE IF NOT EXISTS day14_test (id SERIAL PRIMARY KEY, message TEXT); INSERT INTO day14_test (message) VALUES ('Day 14 persistent data test'); SELECT * FROM day14_test;"
```

The command created a test table, inserted a record, and displayed the stored data.



```
shayan@shayan-VirtualBox:~/day12-compose$ sudo docker compose exec db psql -U day12user -d day12db -c "CREATE TABLE IF NOT EXISTS day14_test (id SERIAL PRIMARY KEY, message TEXT); INSERT INTO day14_test (message) VALUES ('Day 14 persistent data test'); SELECT * FROM day14_test;"
CREATE TABLE
INSERT 0 1
 id |          message
----+-----
  1 | Day 14 persistent data test
(1 row)

shayan@shayan-VirtualBox:~/day12-compose$
```

11. Verify Non-Root Execution

The user running inside the application container was checked using:

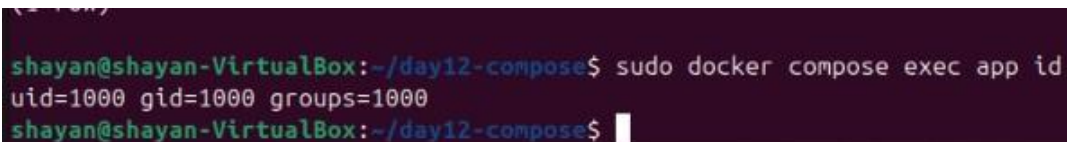
```
sudo docker compose exec app id
```

The output showed:

```
uid=1000
```

```
gid=1000
```

This confirms that the application process is running as a non-root user.



```
shayan@shayan-VirtualBox:~/day12-compose$ sudo docker compose exec app id
uid=1000 gid=1000 groups=1000
shayan@shayan-VirtualBox:~/day12-compose$
```

12. Verify Nginx Reverse Proxy

The application was accessed through Nginx using:

```
curl http://localhost:8080
```

The expected application response confirms that the request successfully passed through Nginx to the application.

Expected response:

Day 12 App: Database connection successful!

```
shayan@shayan-VirtualBox:~/day12-compose$ curl http://localhost:8080
Day 12 App: Database connection successful!shayan@shayan-VirtualBox:~/day12-comp
ose$
```

13. Verify Docker Persistent Volume

The Docker volumes were checked using:

`sudo docker volume ls`

The project volume:

`day12-compose_db-data`

was identified.

The volume details can be inspected using:

`sudo docker volume inspect day12-compose_db-data`

```
shayan@shayan-VirtualBox:~/day12-compose$ sudo docker volume inspect day12-comp
se_db-data
[
  {
    "CreatedAt": "2026-08-31T17:40:01+06:00",
    "Driver": "local",
    "Labels": {
      "com.docker.compose.config-hash": "d45689604e87f7a37a9d41cbb9995727a
0178e79f55e5d410816cb24a799f457",
      "com.docker.compose.project": "day12-compose",
      "com.docker.compose.version": "5.3.1",
      "com.docker.compose.volume": "db-data"
    },
    "Mountpoint": "/var/snap/docker/common/var-lib-docker/volumes/day12-comp
ose_db-data/_data",
    "Name": "day12-compose_db-data",
    "Options": null,
    "Scope": "local"
  }
]
shayan@shayan-VirtualBox:~/day12-compose$
```

14. Image Scanning

The container image was scanned using Trivy to identify HIGH and CRITICAL vulnerabilities.

The scan command used was:

```
sudo trivy image --scanners vuln --severity HIGH,CRITICAL --format table day13-web:1.0
```

The scan identified vulnerabilities in the Debian base image and its installed packages.

The scan result included:

- HIGH vulnerabilities
- CRITICAL vulnerabilities
- Installed package versions
- Available fixed versions where applicable
- Vulnerability identifiers

The scan results were reviewed as part of the container security process.

```
shayan@shayan-VirtualBox:~/day12-compose$ sudo docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE  CR
EATED                                STATUS                                PORTS                                EATED      EATED
day12-compose-app-1                python:3.12-slim                    "sh -c 'pip install ...'"            app        5
minutes ago                        Up 5 minutes (healthy)
day12-compose-db-1                 postgres:16                          "docker-entrypoint.s..."            db          5
minutes ago                        Up 5 minutes (healthy)  5432/tcp
day12-compose-nginx-1              nginx:alpine                         "/docker-entrypoint...."            nginx       5
minutes ago                        Up 5 minutes (unhealthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
shayan@shayan-VirtualBox:~/day12-compose$ sudo docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE  CR
EATED                                STATUS                                PORTS                                EATED      EATED
day12-compose-app-1                python:3.12-slim                    "sh -c 'pip install ...'"            app        6
minutes ago                        Up 6 minutes (healthy)
day12-compose-db-1                 postgres:16                          "docker-entrypoint.s..."            db          6
minutes ago                        Up 6 minutes (healthy)  5432/tcp
day12-compose-nginx-1              nginx:alpine                         "/docker-entrypoint...."            nginx       6
minutes ago                        Up 6 minutes (unhealthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
shayan@shayan-VirtualBox:~/day12-compose$
```